

Examen de mi-semester SRCS Master 1 SAR

Jonathan Lejeune

Mars 2023

2 heures – **Tout document papier interdit**
Tout appareil électronique interdit

- Le barème est sur 21 points et est donné à titre indicatif. La note finale sera bornée à 20 points.
- Dans le code java demandé vous ne gérerez ni les imports ni les exceptions **sauf si c'est explicitement demandé**
- Soyez synthétique et précis dans vos réponses
- Il vous est conseillé de lire attentivement les exercices entièrement avant de composer.
- Lorsque vous ne savez pas répondre à une question, il est préférable de s'abstenir que de répondre une absurdité.
- Dans une question de programmation, n'en faites pas plus que ce qui est demandé (vous n'aurez pas de point en plus).
- Vous trouverez à la fin du sujet, une annexe résumant les API à utiliser.

Exercice 1 – Questions de cours (5 points)

Question 1 $\frac{1}{2}$ point

Qu'affiche le programme java suivant :

```
1 public static void main(String[] args) throws IOException {  
2     try(OutputStream os = new FileOutputStream("/tmp/toto")){  
3         os.write(-1);  
4     }  
5  
6     try(InputStream is = new FileInputStream("/tmp/toto")){  
7         System.out.println(is.read());  
8     }  
9 }
```

Question 2 1 point

Soit une classe Foo qui implante Serializable. L'exécution du code suivant engendre cependant une java.io.NotSerializableException. Expliquer pourquoi et proposer une solution pour corriger ce problème.

```

1 Foo f = new Foo();
2 ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("/tmp/f"));
3 oos.writeObject(f);

```

Question 3 1/2 point

Pourquoi la méthode `readObject()` de la classe `ObjectInputStream` est-elle susceptible de sortir avec une `ClassNotFoundException`¹ ?

Question 4 1/2 point

Quel pattern de programmation l'API des filtres d'I/O respecte-t-elle ?

Question 5 1/2 point

Un processus peut-il ouvrir plusieurs ports d'écoute TCP ? Si non justifier pourquoi, si oui expliquer comment on pourrait mettre en œuvre ceci.

Question 6 1 point

Soit le code suivant :

```

1 public interface Truc {
2     //methodes de Truc
3 }
4 public class Toto implements Truc{
5     //code de la classe Toto
6 }

```

L'instruction `Class<Truc> cl = Toto.class` compile-t-elle ? Si oui préciser pourquoi, si non préciser ce qu'il faudrait corriger.

Question 7 1 point

En programmation concurrente, qu'est ce qu'un moniteur ? Comment sont-ils implantés en java ?

Exercice 2 – Flux d'entrée/sortie avec découpage (6 points)

Sur la plupart des contenus téléchargeables sur internet, il est courant de découper un gros fichier en plusieurs petits fichiers de taille fixe. Chaque fichier de découpage a le même préfixe et est suffixé par l'extension `.parti`, où i désigne le i ème fichier. Le fichier $i + 1$ contient donc la suite du fichier i . La concaténation de l'ensemble des fichiers dans l'ordre de leur indice donne ainsi le même contenu que le gros fichier original. Dans cet exercice, l'indice des fichiers de découpage commence à 1. Le fichier suffixé par `part0` existe mais son contenu se résume à un entier qui est égal au nombre de fichiers de découpage.

Question 1 3 points

Programmer la classe `SplitFileOutputStream` qui étend la classe `OutputStream` et permet d'écrire des données dans un ensemble de fichiers de taille fixe. Elle offre :

- un constructeur prenant deux paramètres : une chaîne de caractères désignant le préfixe des fichiers et la taille maximum d'un fichier.
- la méthode `public void write(int b) throws IOException` qui permet d'écrire un octet dans le fichier courant. Si la taille du fichier courant est égale à la taille maximale passée en paramètre du constructeur, il est nécessaire alors de l'écrire dans le fichier suivant (qui devient le fichier courant).

1. Une réponse qui se contente de dire que "la classe est introuvable" (ou à sens équivalent) est irrecevable

- la méthode `public void close()throws IOException` qui ferme le flux vers le fichier courant. **Tout flux doit être fermé dès que l'on n'utilise plus.**

Exemple d'utilisation avec la méthode `bind` vue en TD/TME :

```
1 try(InputStream is = new FileInputStream("/foo/bar")
2     OutputStream os = new SplitFileOutputStream("/home/bar", 16)){
3     bind(is,os);
4 }
```

Si on considère que le fichier `/foo/bar` est d'une taille de 60 octets, ceci produira 4 fichiers de données (`/home/bar.part1` de 16o, `/home/bar.part2` de 16o, `/home/bar.part3` de 16o et `/home/bar.part4` de 12o) ainsi que le fichier `/home/bar.part0` contenant l'entier 4.

Question 2 3 points

Programmer la classe `SplitFileInputStream` qui étend la classe `InputStream` et qui fait l'opération inverse en lisant un flux d'octets à partir d'un ensemble de fichiers de découpage. Cette classe offre :

- un constructeur prenant un seul paramètre : une chaîne de caractères désignant le préfixe des fichiers.
- la méthode `public int read()throws IOException` qui respecte les mêmes spécifications que n'importe quel `InputStream`
- la méthode `public void close()throws IOException` qui ferme le flux de lecture du fichier courant.

Tout flux doit être fermé dès que l'on n'utilise plus.

Exemple :

```
1 try(InputStream is = new SplitFileInputStream("/home/bar")
2     OutputStream os = new FileOutputStream("/foo/barbis")){
3     bind(is,os);
4 }
```

À l'exécution de ces lignes (et de celles de la question précédente), le fichier `/foo/barbis` doit être identique au fichier `/foo/bar`.

Exercice 3 – MapReduce

La figure 1 schématise un job MapReduce. Le MapReduce est un pattern de calcul parallèle notamment utilisé dans le traitement de données massives.

Un job MapReduce repose sur deux familles de tâche (*Map* et *Reduce*), un espace stockant l'ensemble des données d'entrée du job et un espace pour stocker les données de sortie du job. Les données d'entrée sont découpées en partie indépendantes. Chaque partie est affectée à une tâche de map. Pour chaque donnée lue sur sa partie, une tâche de map appelle une méthode `map` (définie par le programmeur) qui prend en paramètre la donnée et qui renvoie une liste de couple clé-valeur (type `K` pour la clé et `V` pour la valeur). Chaque couple produit par une tâche de map (suite aux appels successifs de la méthode `map`) est aiguillé vers une tâche de reduce. **Tout couple ayant la même clé sera toujours envoyé vers le même reduce.** La politique d'aiguillage est définie par le programmeur par une méthode `dispatch` qui pour une clé et une valeur donnée renvoie l'identifiant de la tâche de reduce destinataire. Lorsque toutes les tâches de map ont terminé leur traitement, les tâches de reduce vont agréger l'ensemble des valeurs associées à une clé donnée. Ensuite, pour chaque

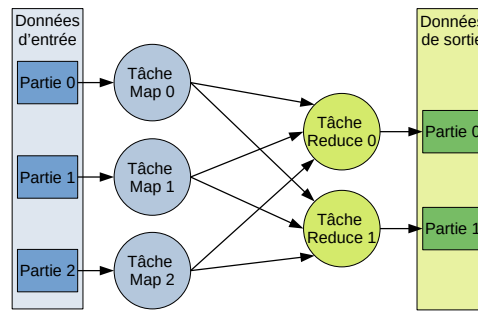


FIGURE 1 – Schéma d'un job MapReduce

clé associée à une liste de valeurs, la tâche appelle la méthode **reduce** (définie par le programmeur) qui prend en paramètre une clé K ainsi qu'une liste de valeurs V et produit une liste de données de sortie. Chaque tâche de reduce écrit l'ensemble de ses données de sortie dans une partie dédiée de l'espace de stockage de sortie du job.

Dans le cadre de cet exercice nous considérerons que les données d'entrée et de sortie sont des lignes de texte (de type **String**).

L'interface `MapReduceJob<K,V>` suivante définit les traitements d'un job MapReduce. Le type K désigne le type des clés des couples transitant entre les maps et les reduces, le type V désigne le type des valeurs.

```

1 public interface MapReduceJob<K,V> {
2     List<Couple<K,V>> map(String line); //définit le traitement d'une donnée pour les tâches de
        maps
3     List<String> reduce(K key, List<V> values); //définit le traitement d'une donnée pour les t
        âches de reduce
4     int dispatch(K k, V v); // définit la politique d'aiguillage en sortie de map. La méthode
        renvoie l'idendifiant du reduce (compris entre 0 et nbreduces -1)
5 }

```

La classe `Couple<L, R>` permet d'agréger deux objets de types potentiellement différents au sein d'un même objet.

```

1 public final class Couple<L, R>{
2     private final L left ;
3     private final R right;
4
5     public Couple(L left, R right) { this.left=left; this.right=right; }
6     public L left() { return left;}
7     public R right() {return right;}
8 }

```

Nous souhaitons compléter les classes suivantes

```

1 public class MapTask<K,V> implements Runnable {
2
3     private final MapReduceJob<K, V> job; //job auquel la tâche est rattachée
4     private final BufferedReader in; //flux d'entrée
5     private final List<ObjectOutputStream> outputs; //liste vers les flux de sortie vers les
        reduces, le flux i correspond à la ième tâche de reduce
6

```

```

7  public MapTask(MapReduceJob<K,V> job, BufferedReader in, List<ObjectOutputStream> outputs)
    {
8      this.in=in;
9      this.job=job;
10     this.outputs=outputs;
11 }
12
13 @Override
14 public void run() {
15     //a compléter
16 }
17
18 }

```

```

1  public class ReduceTask<K,V> implements Runnable {
2
3      private final MapReduceJob<K,V> job;//job auquel la tâche est rattachée
4      private final PrintStream outstream;//flux de sortie
5      private final List<ObjectInputStream> inputs; //listes vers les flux d'entrée depuis les tâches de map, le flux i correspond à la ième tâche de map
6
7      public ReduceTask(MapReduceJob<K,V> job, List<ObjectInputStream> inputs, PrintStream outstream) {
8          this.job=job;
9          this.outstream=outstream;
10         this.inputs=inputs;
11     }
12
13     @Override
14     public void run() {
15         //a compléter
16     }
17
18
19 }

```

Question 1 1 point

Les maps et les reduces vont devoir s'échanger des objets de type **Couple**. Que faut-il modifier à la classe **Couple** pour assurer que le mécanisme de sérialisation de la JVM fonctionne ?

Question 2 1 point

Les tâches de reduce vont lire des couples sur l'ensemble des flux d'entrée en provenance des tâches de map. Exposer une solution pour permettre aux reduces de savoir qu'un flux d'entrée ne comporte plus rien à lire.

Question 3 1½ points

Donner le code de la méthode **run** de la classe **MapTask**. Pour rappel, le flux **BufferedReader** offre une méthode **String readLine()** qui lit une ligne de texte et qui renvoie **null** à la fin du flux. La fermeture des flux n'est pas de la responsabilité de cette classe.

Question 4 3 points

Donner le code de la méthode **run** de la classe **ReduceTask**. Pour rappel le flux **PrintStream** offre une méthode **void println(String)** qui envoie une ligne de texte (à l'instar du flux **System.out**). La fermeture des flux n'est pas de la responsabilité de cette classe. **On souhaite avoir un thread dédié à la lecture de chaque flux d'entrée afin de paralléliser les lectures. Les résultats des lectures seront agrégés dans une variable partagée de type `Map<K,List<V>` qui sera ensuite utilisée pour les appels à la méthode **reduce** du **job**.**

L'exécution d'un tel calcul se fait sur une infrastructure distribuée où chaque tâche de map et de reduce est exécutée dans une JVM dédiée. Une JVM est hébergée sur une des machines physiques de l'infrastructure (notons que le mécanisme de déploiement des JVM sur l'infrastructure n'est pas géré dans cet exercice). Par exemple, le job de la figure 1 déploierait 5 JVM (3 JVM pour les maps et 2 JVM pour les reduces).

La communication entre les tâches de maps et les tâches de reduce vont donc se faire en direct par le biais d'une socket TCP. Les tâches de maps seront les serveurs des tâches de reduce du point de vue de la connexion. Une fois la connexion établie entre une tâche de reduce et une tâche de map, la tâche de reduce envoie son identifiant (`int`) permettant ainsi d'associer une connexion à un identifiant de reduce. Une fois avoir reçu toutes les connexions des reduces il est alors possible d'instancier et d'exécuter une `MapTask`. Une fois que la méthode `run()` de la tâche de map se termine, toutes les sockets de connexions ouvertes doivent être fermées. L'implantation des cas d'erreur du protocole n'est pas demandé.

Question 5 3½ points

Compléter le code suivant qui définit le code d'une JVM de type map.

```

1  public class MapExecutor {
2
3      public static void main(String[] args) throws Exception {
4          int nbreduces = Integer.parseInt(args[0]);
5          String classjob = args[1];
6          String inmaptask = args[2]; //fichier d'entrée du map
7          int port = Integer.parseInt(args[3]); // port d'écoute du map pour que les reduces s'
           y connectent
8          MapReduceJob<?, ?> job = makeJob(classjob); // construction du job à partir du nom de
           sa classe
9          deployMapTask(nbreduces, inmaptask, port, job); // déploiement et exécution de la tâ
           che de map sur la JVM locale
10     }
11
12     //construit une instance de job a partir du nom de sa classe.
13     //on suppose que toute classe implantant MapReduceJob définit un constructeur
14     //sans argument
15     public static MapReduceJob<?,?> makeJob(String classjob) throws Exception{
16         // A COMPLETER
17     }
18
19
20     public static void deployMapTask(int nbreduces, String inmaptask, int port,
           MapReduceJob<?,?> job) throws Exception {
21         // A COMPLETER
22     }

```

Java I/O classes

Abstract Classes	
Class	Main public fields
abstract class InputStream implements Closeable	• abstract int read() throws IOException • int read(byte b[]) throws IOException • int read(byte b[], int off, int len) throws IOException • void close() throws IOException
abstract class OutputStream implements Closeable, Flushable	• abstract void write(int b) throws IOException • void write(byte b[]) throws IOException • void write(byte b[], int off, int len) throws IOException • void flush() throws IOException • void close() throws IOException

Main InputStream sub-classes	
Class	Main public fields
class FileInputStream extends InputStream	• public FileInputStream(String name) throws FileNotFoundException • public FileInputStream(File file) throws FileNotFoundException
class ByteArrayInputStream extends InputStream	• public ByteArrayInputStream(byte buf[]) • public ByteArrayInputStream(byte buf[], int offset, int length)
class BufferedInputStream extends FilterInputStream	• public BufferedInputStream(InputStream in)
class DataInputStream extends FilterInputStream implements DataInput	• public DataInputStream(InputStream in) • String readLine() throws IOException • double readDouble() throws IOException • float readFloat() throws IOException • String readUTF() throws IOException • long readLong() throws IOException • int readInt() throws IOException • char readChar() throws IOException • boolean readBoolean() throws IOException • byte readByte() throws IOException
class ObjectInputStream extends InputStream implements ObjectInput, ObjectStreamConstants	• public ObjectInputStream(InputStream in) throws IOException • methods of DataInputStream • final Object readObject() throws IOException, ClassNotFoundException
class PipedInputStream extends InputStream	• public PipedInputStream(PipedOutputStream src) throws IOException • public PipedInputStream() • void connect(PipedOutputStream src) throws IOException

Main OutputStream sub-classes	
Class	Main public fields
class FileOutputStream extends OutputStream	• public FileOutputStream(String name) throws FileNotFoundException • public FileOutputStream(File file) throws FileNotFoundException
class ByteArrayOutputStream extends OutputStream	• public ByteArrayOutputStream() • public ByteArrayOutputStream(int size) • public byte[] toByteArray()
class BufferedOutputStream extends FilterOutputStream	• public BufferedOutputStream(OutputStream out)

class DataOutputStream extends FilterOutputStream implements DataOutput	• public DataOutputStream(OutputStream out) • void writeDouble(double v) throws IOException • void writeFloat(float v) throws IOException • void writeUTF(String str) throws IOException • void writeLong(long v) throws IOException • void writeInt(int v) throws IOException • void writeChar(int v) throws IOException • void writeBoolean(boolean v) throws IOException • void writeByte(int v) throws IOException • void writeBytes(String s) throws IOException
class ObjectOutputStream extends OutputStream implements ObjectOutput, ObjectOutputStreamConstants	• public ObjectOutputStream(OutputStream out) throws IOException • methods of DataOutputStream • void writeObject(Object obj) throws IOException
class PipedOutputStream extends OutputStream	• public PipedOutputStream(PipedInputStream snk) throws IOException • public PipedOutputStream() • void connect(PipedInputStream snk) throws IOException